

```
int score = 90;
```

기본 문법: 자료형, 변수, 연산자

C++ 프로그래밍 2장 · 저장하고, 계산하고, 입력받기

```
std::cin >> age;
```

이 장의 학습 목표



01

자료형의 종류를 알고 상황에 맞게 고를 수 있다.



02

변수와 상수를 선언하고 값을 저장·변경할 수 있다.



03

연산자로 계산식과 조건식을 만들 수 있다.



04

형 변환을 이해하고 안전하게 사용할 수 있다.



05

키보드로 입력받고 원하는 형식으로 출력할 수 있다.

미리 알아 둘 용어

데이터

프로그램이 다루는 값. 숫자, 글자, 참/거짓 등

메모리

실행 중 데이터를 잠시 보관하는 작업 공간

자료형

데이터의 종류. 정수·소수·글자인지를 정함

변수

데이터를 담는 이름 붙은 상자. 내용물을 바꿀 수 있음

상수

한 번 정하면 바꿀 수 없는 값

연산자

+, -, == 처럼 계산이나 비교를 하는 기호

바이트(byte)

메모리 크기 단위. 1바이트 = 8비트



2.1

기본 자료형

자료형이 왜 필요할까

그릇

그릇 상자

옷

옷 상자

책

책 상자

이사할 때 물건마다 알맞은 상자가 있듯, 데이터도 종류에 따라 알맞은 크기와 모양의 저장 공간이 필요합니다. 이 "상자의 종류"가 자료형(data type)입니다.

자료형을 정해 주면 컴퓨터는...



메모리를 얼마나 쓸지 안다



그 안의 0과 1을 어떻게 해석할지 안다

C++의 기본 자료형 한눈에 보기

분류	자료형	크기	담을 수 있는 값	예시
정수	short	2바이트	약 -3만 2천 ~ 3만 2천	나이, 층수
정수	int	4바이트	약 -21억 ~ 21억	점수, 개수 (가장 많이 씀)
정수	long long	8바이트	약 ± 900 경	인구수, 큰 금액
실수	float	4바이트	소수점 아래 약 7자리 정밀도	간단한 그래픽 좌표
실수	double	8바이트	소수점 아래 약 15자리 정밀도	키, 몸무게, 평균 (실수의 기본)
문자	char	1바이트	글자 1개 (영문·숫자·기호)	'A', '7', '?'
논리	bool	1바이트	true 또는 false	합격 여부, 로그인 여부

크기는 일반적인 PC 기준이며, 환경에 따라 다를 수 있습니다.

처음에는 이 네 가지면 충분합니다

int

정수

```
int age = 20;
```

나이: 20

double

실수

```
double height = 175.3;
```

키: 175.3cm

char

글자 하나

```
char grade = 'A';
```

학점: A

bool

예/아니오

```
bool isStudent = true;
```

학생인가요? 1

bool을 그냥 출력하면 1/0으로 나옵니다. std::boolalpha를 먼저 출력하면 true/false로 나옵니다. · 큰 수는 long long 5100000000LL처럼 씁니다.

주의할 두 가지: 실수와 문자



실수는 근삿값이다

컴퓨터는 0.1 같은 수를 정확히 저장하지 못하고 아주 가까운 값으로 저장합니다.

그래서 $0.1 + 0.2 == 0.3$ 을 비교하면 거짓이 나올 수 있습니다.

돈 계산처럼 정확해야 하는 곳에서는 "원" 단위 정수로 계산합니다.

```
char grade = 'A';  
char next = grade + 1; // 66 → 'B'  
static_cast<int>(grade) // 65
```

실행 결과

학점: A 다음 글자: B
'A'의 번호: 65

글자는 사실 숫자(아스키 코드)로 저장됩니다. 한글은 char에 담을 수 없어 std::string을 씁니다.

sizeof로 크기 확인하기

```
std::cout << "char      : "  
          << sizeof(char) << "바이트\n";  
// bool, int, long long, double도  
// 같은 방식으로 출력
```

실행 결과

```
char      : 1바이트  
bool      : 1바이트  
int       : 4바이트  
long long : 8바이트  
double    : 8바이트
```

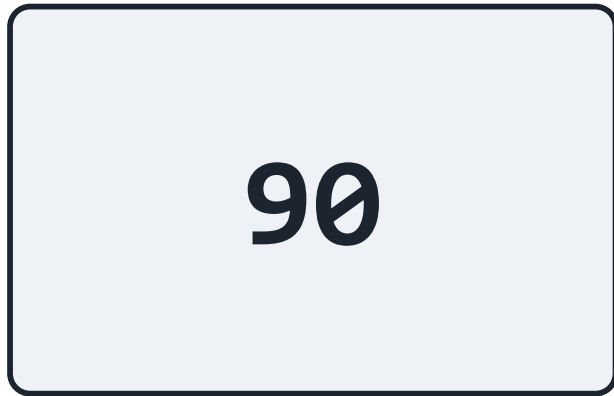
sizeof는 자료형이나 변수가 메모리를 몇 바이트 차지하는지 알려 줍니다. 컴퓨터 환경에 따라 크기가 조금 다를 수 있으니 직접 확인해 보세요. (예제 2-5)

2.2

변수와 상수

변수 = 이름 붙은 상자

`int score`



자료형 + 이름표 + 값

```
int score;    // 선언: 상자를 만들고 이름표를 붙임
score = 90;   // 대입: 상자에 90을 넣음
score = 95;   // 다시 대입: 90은 사라지고 95
```



= 는 "같다"가 아니다

C++에서 =는 "오른쪽 값을 왼쪽 변수에 넣는다"는 뜻입니다. 같은지 비교할 때는 ==를 씁니다(2.3절).

변수의 값은 바뀐다

```
int coins = 100;
std::cout << "처음 동전: " << coins << "\n";
coins = 250;
std::cout << "동전을 모은 후: " << coins << "\n";
coins = coins - 50; // 현재 값에서 50을 빼서 다시 넣음
std::cout << "아이템 구매 후: " << coins << "\n";
```

실행 결과

처음 동전: 100
동전을 모은 후: 250
아이템 구매 후: 200

100

처음



250

대입



200

250 - 50

초기화: 만들면서 바로 값 넣기

```
int a = 10;
```

방법 1: 대입 형태

```
int b(10);
```

방법 2: 괄호 형태

```
int c{10};
```

방법 3: 중괄호 형태 (권장)

```
int x = 3.7; // 허용됨. x에는 3 (눈치채기 어려움)  
int y{3.7}; // 컴파일 오류! 값 손실을 미리 알려 줌
```

중괄호 초기화는 값이 잘려 나가는 실수를 컴파일러가 막아 줍니다.



가장 많이 하는 실수

int total;처럼 값을 넣지 않고 바로 쓰면, 이전에 메모리에 남아 있던 알 수 없는 값(쓰레기 값)이 들어 있습니다.
변수는 만들 때 항상 초기화하세요.

변수 이름 짓는 규칙

규칙	올바른 예	잘못된 예
영문자, 숫자, 밑줄(_)만 사용	score1, max_value	score!, my-name
숫자로 시작할 수 없음	player2	2player
공백을 넣을 수 없음	totalScore	total score
C++ 키워드는 쓸 수 없음	myInt	int, return
대소문자를 구분함	age와 Age는 다른 변수	



좋은 이름

무엇을 담는지 바로 알 수 있는 이름
a, b보다 price, count

studentCount
카멜 표기법
student_count
스네이크 표기법

상수와 auto

```
const double PI = 3.14159;
double radius = 5.0;
double area = PI * radius * radius;
// PI = 3.0; // 주석을 풀면 컴파일 오류
```

실행 결과

반지름 5인 원의 넓이: 78.5397

바뀌면 안 되는 값은 const로 상수로 만듭니다. 선언할 때 값을 정해야 합니다. (예제 2-7)

constexpr

컴파일할 때 이미 값이 확정된 상수
고정된 숫자 → constexpr, 실행 중에 정해지는 값 → const

```
constexpr int MONTHS_PER_YEAR = 12;
```

auto

초기값을 보고 컴파일러가 자료형을 정함
auto count = 10; → int
auto rate = 0.25; → double
지금은 자료형을 직접 쓰는 연습부터 (15장에서 활용)

2.3

연산자

산술 연산자

연산자	의미	예	결과
+	더하기	17 + 5	22
-	빼기	17 - 5	12
*	곱하기	17 * 5	85
/	나누기	17 / 5	3
%	나머지	17 % 5	2

정수 나눗셈

$$17 / 5 = 3$$

정수끼리 나누면 소수점 아래를 버리고 몫만 남습니다.

17.0 / 5 처럼 한쪽을 실수로 만들면 3.4



0으로 나누면 프로그램이 비정상 종료될 수 있으니 주의하세요.

나머지 연산자 % 활용: 초를 시·분·초로

```
int seconds = 3725;
int hours = seconds / 3600;           // 1시간 = 3600초
int minutes = (seconds % 3600) / 60; // 남은 초를 분으로
int remain = seconds % 60;           // 마지막 남은 초
```

실행 결과

3725초는 1시간 2분 5초입니다.

% 는 이럴 때 씁니다

- 짝수·홀수 판별: $n \% 2$ 가 0이면 짝수
- 시간·단위 변환
- 자릿수 분리 (연습문제)
- 거스름돈 계산 (종합 예제)

복합 대입 연산자

줄여 쓴 형태	같은 뜻
<code>x += 3</code>	<code>x = x + 3</code>
<code>x -= 3</code>	<code>x = x - 3</code>
<code>x *= 3</code>	<code>x = x * 3</code>
<code>x /= 3</code>	<code>x = x / 3</code>
<code>x %= 3</code>	<code>x = x % 3</code>

자기 자신에게 계산한 결과를 다시 넣을 때 짧게 씁니다.

용돈 기입장 (예제 2-10)

`int money = 10000;` **10000원**

`money += 5000;` **15000원**

`money -= 3000;` **12000원**

`money *= 2;` **24000원**

`money /= 4;` **6000원**

증감 연산자: 전위와 후위

n++

후위 증가

현재 값을 먼저 쓰고, 그다음 1 늘림

```
int n = 5;  
int r1 = n++;
```

r1 = 5, n = 6

++m

전위 증가

먼저 1 늘리고, 늘어난 값을 사용

```
int m = 5;  
int r2 = ++m;
```

r2 = 6, m = 6



헛갈리면 `count++`; 처럼 한 줄에 단독으로 쓰세요. 그러면 전위와 후위의 결과가 같습니다.

비교 연산자와 논리 연산자

연산자	의미	x=5, y=10	결과
<code>==</code>	같다	<code>x == y</code>	false
<code>!=</code>	다르다	<code>x != y</code>	true
<code><</code>	작다	<code>x < y</code>	true
<code>></code>	크다	<code>x > y</code>	false
<code><=</code>	작거나 같다	<code>x <= 5</code>	true
<code>>=</code>	크거나 같다	<code>x >= y</code>	false

연산자	이름	의미
<code>&&</code>	AND(그리고)	양쪽 모두 참이어야 참
<code> </code>	OR(또는)	하나만 참이어도 참
<code>!</code>	NOT(아니다)	참과 거짓을 뒤집음



가장 흔한 실수

if (x = 10)은 x에 10을 넣어 버립니다. 같은지 비교는 반드시 `==`를 쓰세요.

예제: 영화관 입장 조건

```
int age = 25;
bool hasTicket = true;
std::cout << std::boolalpha;
(age >= 19 && hasTicket)    // 성인이고 표가 있다
(age < 13 || age >= 65)     // 어린이 또는 경로
!hasTicket                  // 표가 없다
```

실행 결과

```
청소년 관람불가 영화 입장 가능? true
할인 대상(어린이 또는 경로)? false
표가 없나요? false
```



그리고

둘 다 참 → 참



또는

하나라도 참 → 참



아니다

참 ↔ 거짓

연산자 우선순위

순위	연산자	설명
1 (가장 먼저)	()	괄호
2	++ -- !	단항 연산자
3	* / %	곱셈, 나눗셈, 나머지
4	+ -	덧셈, 뺄셈
5	< <= > >=	크기 비교
6	== !=	같음 비교
7	&&	논리 AND
8		논리 OR
9 (가장 나중)	= += -= 등	대입

2 + 3 * 4 // 14 (곱셈 먼저)
 (2 + 3) * 4 // 20 (괄호 먼저)



헛갈리면 괄호를 치세요

우선순위를 모두 외울 필요는 없습니다. 괄호는 계산 순서를 확실히 하고, 읽는 사람도 편하게 해 줍니다.

2.4

형 변환

자동 형 변환



큰 컵의 물을 작은 컵에 부으면 넘치듯, 형 변환 중에는 값의 일부가 사라질 수 있습니다.

```
5 + 2.5    // int 5 → 5.0으로 바뀌어 7.5  
'A' + 1    // char 'A' → 65로 바뀌어 66
```

```
double d = 3.99;  
int i = d; // i는 3 (반올림이 아님!)
```

변환 방향	결과	안전성
int → double	5 → 5.0	안전
char → int	'A' → 65	안전
double → int	3.99 → 3	소수점 손실
long long → int	범위 밖 값은 엉뚱하게	위험

서로 다른 자료형끼리 계산하면 더 넓은 범위의 자료형 쪽으로 맞추집니다.

명시적 형 변환: static_cast

```
int sum = kor + eng + math; // 90 + 85 + 78
double avg1 = sum / 3; // 잘못
double avg2 = static_cast<double>(sum) / 3; // 올바른
```

실행 결과

```
총점: 253
잘못된 평균: 84
올바른 평균: 84.3333
```

왜 avg1은 84일까?

sum / 3은 정수끼리의 나눗셈이라 먼저 84로 계산되고, 그 84가 double에 84.0으로 저장됩니다.

결과를 double에 담는다고 계산 자체가 실수로 바뀌지는 않습니다.

계산하기 전에 static_cast<double>로 바꿔야 합니다.

오버플로: 범위를 넘으면 생기는 일

```
int big = 2147483647; // int의 최댓값
big = big + 1;
std::cout << big << "\n";
```

실행 결과

-2147483648

99999 → 00000

자동차 주행 거리계가 넘어가듯 값이 한 바퀴 돌아 음수가 되었습니다. 많은 컴퓨터에서 이렇게 보이지만, C++ 표준은 결과를 보장하지 않는 위험한 동작입니다.

다른 형 변환 연산자	용도	배우는 곳
<code>const_cast</code>	const 속성을 잠시 없앴	5장
<code>reinterpret_cast</code>	메모리를 다른 자료형으로 해석 (위험)	5장
<code>dynamic_cast</code>	상속 관계 객체를 안전하게 변환	10장

2.5

표준 입출력

cout과 cin: 화살표 방향으로 흐른다



`std::cout`

프로그램 → 화면 (출력)

화살표가 cout 쪽으로 흘러감



`std::cin`

키보드 → 변수 (입력)

화살표가 변수 쪽으로 흘러감

이스케이프	의미
<code>\n</code>	줄 바꿈
<code>\t</code>	탭(일정 간격)
<code>\"</code>	큰따옴표 출력
<code>\\</code>	백슬래시 출력

₩로 시작하는 특수 문자를 이스케이프 시퀀스라고 합니다.

```
int level = 7;
std::cout << "현재 레벨: " << level << "\n";
```

std::cin으로 입력받기

```
int a = 0;
int b = 0;
std::cout << "첫 번째 수를 입력하세요: ";
std::cin >> a;
std::cout << "두 번째 수를 입력하세요: ";
std::cin >> b;
```

실행 결과 (12, 30은 사용자 입력)

```
첫 번째 수를 입력하세요: 12
두 번째 수를 입력하세요: 30
12 + 30 = 42
```

```
double celsius = 0.0;
std::cin >> celsius;
double fahrenheit =
    celsius * 9.0 / 5.0 + 32.0;
```

실행 결과

```
섭씨 온도를 입력하세요: 25
섭씨 25도는 화씨 77도입니다.
```

std::cin >> a >> b; 처럼 이어 쓰면 "12 30"을 한 줄에 입력해도 됩니다.

문자열 입력: std::string과 getline

```
std::string name;  
std::string address;  
std::cin >> name;           // 빈칸 전까지만  
std::cin.ignore();           // 남은 Enter 버리기  
std::getline(std::cin, address); // 한 줄 전체
```

실행 결과

```
이름: 홍길동  
주소: 서울시 종로구 세종대로 1  
홍길동님의 주소는 서울시 종로구 세종대로 1입니다.
```



왜 cin.ignore()가 필요할까?

이름을 입력하고 Enter를 누르면 cin >>은 이름만 가져가고 Enter(줄 바꿈)는 남겨둡니다.

그대로 getline을 부르면 남은 Enter를 "빈 줄"로 읽고 바로 끝나 버립니다.

ignore()가 그 Enter를 치워 줍니다.

입력 오류와 출력 형식

```
std::cin >> age;
std::cout << std::boolalpha;
std::cout << std::cin.fail();
```

실행 결과

나이를 입력하세요: abc
입력 실패 여부: true
저장된 값: 0

숫자 대신 "abc"를 넣으면 cin은 실패 상태가 되고 변수에는 0이 들어갑니다. 다시 입력받는 방법은 3장에서 배웁니다.

<iomanip> 도구	의미
<code>std::fixed</code>	일반 소수점 형식으로 출력
<code>std::setprecision(n)</code>	fixed와 함께: 소수점 아래 n자리 (반올림)
<code>std::setw(n)</code>	다음 값을 n칸 너비에 오른쪽 정렬

예제 2-19 실행 결과

가격: 1234.57
[42]

C++20 std::format: `std::format("평균: {:.1f}점", 84.3333) → 평균: 84.3점`

종합

종합 예제

종합 예제 2-A: BMI 계산기

1 입력

이름, 키, 몸무게를
cin >>으로 받기

2 계산

키를 m로 바꾸고
 $BMI = \text{몸무게} \div \text{키}^2$
18.5 이상 23 미만
→ 정상

3 출력

소수점 한 자리,
true/false로 출력

실행 결과

이름을 입력하세요: 김철수
키(cm)를 입력하세요: 175
몸무게(kg)를 입력하세요: 68

== 김철수님의 건강 정보 ==
키 : 175.0 cm
몸무게 : 68.0 kg
BMI : 22.2
정상 범위: true

입력 → 계산 → 출력 세 덩어리로 나누고 주석을 달면 읽기 쉽고 고치기도 쉽습니다.

종합 예제 2-B: 거스름돈 계산기

```
const int PRICE = 4300;
int change = paid - PRICE;      // 5700
int won1000 = change / 1000;    // 5
change %= 1000;                 // 700
int won500 = change / 500;      // 1
change %= 500;                  // 200
int won100 = change / 100;      // 2
```

실행 결과

낸 돈을 입력하세요: 10000
거스름돈: 5700원

5700원을 나누면



1000원 지폐

5장



500원 동전

1개



100원 동전

2개

몫(/)으로 개수를, 나머지(%)로 남은 금액을 구합니다.

정리

핵심 정리와 연습문제

핵심 정리



기본은 int, double, char, bool



변수는 만들 때 항상 초기화, 중괄호 {}가 가장 안전



바뀌면 안 되는 값은 const / constexpr



=는 대입, ==는 비교. 정수끼리 나누면 소수점 버림



헷갈리는 계산 순서는 괄호로



형 변환은 static_cast<자료형>(값)



cin >>으로 입력, 띄어쓰기 있는 문장은 getline

확인 문제

1. 알맞은 자료형: ① 학생 수 ② 키(cm) ③ 혈액형 한 글자 ④ 출석 여부
2. 올바른 변수 이름: myAge, 2nd, total_sum, float, user name
3. 결과: ① $20 / 6$ ② $20 \% 6$ ③ $20.0 / 8$ ④ $2 + 3 * 4 - 1$
4. `int x = 10; int y = x++;` 후 x와 y의 값은?
5. `double avg = 7 / 2;`가 3.5가 아닌 이유와 고치는 법
6. `std::cin >>`과 `std::getline`의 차이는?

7. 잘못된 곳 세 군데를 찾아 고치세요.

```
const double RATE = 0.1;
int 1stPrice = 20000;
RATE = 0.2;
std::cout << 1stPrice * RATE
           << "\n"
```

프로그래밍 과제



1. 사칙연산 계산기

두 정수로 +, -, *, /(소수점 둘째 자리), % 출력



2. 원의 둘레와 넓이

반지름 입력, π 는 constexpr 상수



3. 시간 변환기

분을 "O일 O시간 O분"으로 (1505분 → 1일 1시간 5분)



4. 성적 처리

이름과 세 과목 → 총점, 평균(소수점 첫째 자리), 60점 이상 여부(bool)



5. (도전) 자릿수 분리

세 자리 정수의 백·십·일의 자리 출력 (힌트: / 와 %)

```
if (bmi < 18.5) { ... }
```

다음 장 미리 보기

3장 제어문: 조건문으로 상황에 따라 다른 일을 하고, 반복문으로 같은 일을 여러 번 합니다.
BMI 계산기도 "저체중/정상/과체중"을 판정하도록 발전시킵니다.